Vietnamese-German University

PROJECT REPORT

SMART DESK CAMERA

Updated Report Based on the Revised Hardware and Working Mechanism

Course:	Information Technology
Course Code:	60SCI027
Instructor:	Dr. Vuong Quoc Bao
University:	Vietnamese-German University (VGU)
Date:	2/04/2026

Student Name(s)

1. Nguyễn Minh Khôi - 10425085
2. Ngô Anh Khôi - 10425136
3. Đậu Phi Việt Khoa - 10425078
4. Nguyễn Quốc Thiên - 10425106

Vietnamese-German University

Abstract

The ESP32-S3 is used primarily as a Wi-Fi camera node that captures images through the OV2640 camera and exposes them through an HTTP endpoint. Then, a browser-based user interface running on a computer records the user's voice, retrieves frames from the ESP32-S3, sends both image and audio data to a FastAPI backend, and plays the generated response through the computer speaker. The backend integrates OpenAI services for speech transcription, visual understanding, and text-to-speech generation. This architecture simplifies the embedded hardware, reduces pin conflicts, and preserves the system's smart interactive behavior.

Table of Contents

Section	Title	Page
Chapter 1	Introduction	2
Chapter 2	Team Members	4
Chapter 3	Project Timeline	4
Chapter 4	System Overview	4
Chapter 5	Hardware Design	5
Chapter 6	Software Design	6
Chapter 7	Implementation	8
Chapter 8	Testing and Results	8
Chapter 9	Advantages, Limitations, and Future Improvements	10
Chapter 10	Conclusion	10
Chapter 11	References	10

Chapter 1. Introduction

Background

Smart assistants are increasingly built from low-cost embedded hardware combined with cloud-based intelligence. Although that design showed the project direction clearly, it also introduced hardware complexity because the camera already occupied many pins, leaving limited room for stable audio integration. The revised version keeps the ESP32-S3 focused on what it performs well in this prototype: camera capture and wireless communication.

Objective

- To redesign the Smart Desk Camera so that the ESP32-S3 works as a dedicated camera node.

- To maintain image-based and voice-based interaction while simplifying the embedded hardware.
- To build a browser interface that handles live preview, microphone recording, and playback of the AI response.
- To connect the browser interface and the ESP32-S3 to a backend server over the local network.
- To process the collected image and audio data using cloud AI services and return a useful spoken answer.

Scope

- ESP32-S3 camera initialization and Wi-Fi communication
- HTTP-based image capture endpoint on the ESP32-S3
- Browser-based live preview and voice recording
- FastAPI backend for request handling
- AI-based speech transcription, visual reasoning, and text-to-speech response
- Playback of the generated answer on the user computer

Chapter 2. Team Members

Name	ID	Email	Role
Nguyễn Minh Khôi (Leader)	10425085	10425085@student.vgu.edu.vn	Hardware design, report writing
Ngô Anh Khôi	10425136	10425136@student.vgu.edu.vn	Software design, testing
Đậu Phi Việt Khoa	10425078	10425078@student.vgu.edu.vn	Software design, testing
Nguyễn Quốc Thiên	10425106	10425106@student.vgu.edu.vn	Hardware design, software design

Chapter 3. Project Timeline

Week	Activity
Week 1 (30/1 to 6/2)	Topic selection and labor division
Week 2 (6/2 to 13/2)	Brainstorming project approach
Week 3 (13/2 to 20/2)	Testing project approach
Week 4 (20/2 to 27/2)	Buying and gathering hardware components
Week 5 (27/2 to 6/3)	Creating the hardware
Week 6 (6/3 to 13/3)	Creating the software
Week 7 (13/3 to 20/3)	Revising integration approach and debugging
Week 8 (20/3 to 27/3)	Frontend and backend integration
Week 9 (27/3 to 3/4)	Report writing and product testing

Chapter 4. System Overview

General Description

The Smart Desk Camera is a distributed prototype consisting of three main parts: the ESP32-S3 camera node, a browser-based user interface running on a computer, and a FastAPI backend server. The ESP32-S3 captures JPEG frames and serves them over Wi-Fi. The browser provides the live preview, records the user's voice through the computer microphone, and sends the captured image together with the recorded audio to the backend. The backend then performs AI processing and returns an audio response that is played through the computer speaker.

System Workflow

- The ESP32-S3 connects to the local Wi-Fi network and starts a web server.
- The browser interface requests frames from the /capture endpoint to display a live preview or a still image.
- When the user holds the interaction button in the browser, the browser records audio from the computer microphone.
- The browser sends the selected image frame and the recorded audio file to the FastAPI backend through a multipart HTTP request.
- The backend transcribes the audio, uses the image and the transcript to generate an answer, synthesizes speech, and returns the audio response.
- The browser plays the returned audio through the computer speaker and optionally shows the transcript and AI answer in the interface.

Core Functions

- Image capture from the OV2640 camera
- HTTP image serving from the ESP32-S3
- Live preview and manual interaction from a browser interface
- Microphone recording and audio playback on the computer
- Backend processing using OpenAI APIs
- Concise AI-generated spoken responses for desk-related questions

Chapter 5. Hardware Design

Hardware Components

- ESP32-S3 development board
- OV2640 camera module
- Jumper wires and power supply
- Computer connected to the same network for the web interface, microphone input, and speaker output

Pin Configuration of the ESP32-S3 Camera Node

Signal	GPIO / Value
SIOD (SDA)	GPIO4
SIOC (SCL)	GPIO5
Y2	GPIO11
Y3	GPIO9
Y4	GPIO8
Y5	GPIO10
Y6	GPIO12
Y7	GPIO18
Y8	GPIO17
Y9	GPIO16
VSYNC	GPIO6
HREF	GPIO7
PCLK	GPIO13
XCLK	GPIO15
PWDN	-1
RESET	-1

Design Considerations

By keeping the ESP32-S3 focused on camera operation and Wi-Fi serving, the system avoids many of the pin conflicts and timing issues that arose when audio peripherals were attached to the board. The code also uses conservative camera settings by default and upgrades to higher resolution only when PSRAM is detected, which helps prevent memory-related initialization failures.

Chapter 6. Software Design

Embedded Software

The embedded software is written in Arduino-style C++ for the ESP32-S3. Its main responsibilities are camera configuration, Wi-Fi connection, web server initialization, and HTTP image delivery. The /capture endpoint returns a JPEG image frame whenever the browser requests one. To improve browser compatibility, the firmware also sends a permissive CORS header so the locally hosted web app can fetch the image directly.

Frontend Software

The user interface is implemented as an HTML page with JavaScript and Tailwind CSS styling. This interface provides live view, device settings, a press-and-hold interaction button, status indicators, and chat-like feedback bubbles. JavaScript manages both image polling from the ESP32-S3 and microphone recording through the browser media APIs. It also receives the generated audio reply from the backend and plays it on the computer speaker.

Server Software

The backend is built with FastAPI in Python. It accepts an uploaded image and audio file through the /process endpoint, then runs a three-stage pipeline: speech transcription, image-aware response generation, and text-to-speech synthesis. The resulting audio stream is returned to the browser, while the recognized transcript and AI answer are exposed through HTTP headers for display in the interface.

Functional Workflow Summary

Stage	Description
1. Camera service	ESP32-S3 initializes the OV2640 camera and joins the Wi-Fi network.
2. Preview	Browser fetches one frame or a repeated series of frames from /capture.
3. Voice capture	User holds the on-screen button and the browser records audio.
4. Upload	Browser posts image + audio to the FastAPI backend as multipart form data.
5. AI processing	Backend transcribes speech, analyzes the image with the prompt, and creates a text answer.
6. Response	Backend synthesizes speech and sends the audio reply back to the browser.
7. Playback	Browser plays the answer and shows transcript/response text in the UI.

Chapter 7. Implementation

Sector	Methods
Camera integration	The OV2640 camera is connected to the ESP32-S3 through the esp_camera library. The firmware configures SCCB communication, synchronization pins, pixel clock, frame size, JPEG quality, and framebuffer count.
Memory handling	The firmware starts with safe defaults such as QVGA and one framebuffer. When PSRAM is detected, it upgrades to SVGA resolution, better JPEG quality, and two framebuffers for improved image quality.
HTTP serving	A WebServer instance exposes the /capture route. On every request, the board acquires a frame buffer, sends the JPEG bytes to the client, and then releases the frame buffer safely.
Browser interaction	The HTML/JavaScript frontend handles the live view loop, microphone access, audio recording, visual status updates, quick AI actions, and playback of backend audio responses.
Backend processing	The FastAPI server uses OpenAI services to transcribe audio, answer the user based on the uploaded image and spoken question, and synthesize the answer as speech.
System integration	The final implementation separates sensing on the ESP32-S3 from interaction and audio on the computer. This division makes debugging easier because network, camera, frontend, and backend issues can be isolated more clearly.

Chapter 8. Testing and Results

Unit Testing

- Camera initialization test on ESP32-S3
- Wi-Fi connection and IP address verification
- HTTP /capture endpoint test
- Browser live preview test

- Microphone permission and recording test in the browser
- FastAPI endpoint test with uploaded audio and image files
- Audio playback test on the computer speaker

Integration Testing

After the individual modules were validated, the full pipeline was tested from end to end. The ESP32-S3 delivered camera frames successfully, the browser displayed the preview, user speech was recorded, and the backend processed the combined request. The final output was an audio reply played through the computer speaker, with optional text feedback visible in the web interface.

Observed Results

- Successful initialization of the ESP32-S3 camera service
- Successful image retrieval from the /capture endpoint over the local network
- Stable browser-based voice recording and playback compared with the old embedded-audio setup
- Successful FastAPI processing of combined image and audio requests
- Working AI response flow using speech-to-text, image understanding, and text-to-speech
- Improved reliability because audio peripherals were removed from the embedded node

Issues Encountered

- Camera initialization may still fail if memory settings or PSRAM configuration are incorrect.
- Live preview depends on stable Wi-Fi connectivity and correct IP configuration.
- Browser microphone permissions must be granted before interaction can start.
- The full system still relies on backend availability and API access for AI processing.
- Real-time performance depends on network latency, browser performance, and backend response time.

Chapter 9. Advantages, Limitations, and Future Improvements

Advantages	Limitations	Possible Improvements
Simpler embedded hardware design	Still dependent on a computer for audio interaction	Move toward a dedicated frontend device or a more integrated embedded platform
Reduced GPIO conflict and easier debugging	System remains dependent on local network stability	Improve connection handling and add clearer reconnection logic
Flexible browser-based user interaction	Live preview uses repeated frame requests rather than a true video stream	Adopt a more efficient streaming method in future versions
Powerful AI processing without heavy edge computation	Cloud/backend dependency increases latency and requires API access	Optimize prompts, compress uploads, or explore hybrid on-device preprocessing
Modular architecture with separable components	Prototype is suitable for demonstration but not yet production-ready	Add authentication, logging, better error recovery, and cleaner packaging. Add speaker and microphone.

Chapter 10. Conclusion

The design uses the embedded board as a dedicated camera node and transfers microphone, speaker, and user control functions to a browser-based interface on the computer. This change reduces hardware complexity, avoids several pin-mapping issues, and produces a more reliable end-to-end workflow. The final system demonstrates that a low-cost ESP32-S3 camera can be combined with a local web application, a Python backend, and cloud AI services to create an interactive smart desk assistant. The project also provides a clear foundation for future improvements such as better streaming, tighter security, and more polished user interaction.

Chapter 11. References

Espressif Systems. (n.d.). ESP32-S3 series datasheet. https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/

Espressif Systems. (n.d.). Arduino-ESP32 documentation. https://docs.espressif.com/projects/arduino-esp32/

Espressif Systems. (n.d.). ESP32 Camera Driver (esp32-camera). https://github.com/espressif/esp32-camera

FastAPI. (n.d.). FastAPI documentation. https://fastapi.tiangolo.com/

OpenAI. (n.d.). Audio API reference. https://platform.openai.com/docs/api-reference/audio

OpenAI. (n.d.). Speech-to-text guide. https://platform.openai.com/docs/guides/speech-to-text

OpenAI. (n.d.). Text-to-speech guide. https://platform.openai.com/docs/guides/text-to-speech